

Autora: Camila Gois de Jesus
Curso: Pós-Graduação em IA e Ciência de Dados
Instituição: SENAI/SC
Disciplina: IA Embarcada e Modelos Compactos
Professor: Rodrigo Kobashikawa
Ano: 2026

SMART FARMING INT8

Classificação termo-higrométrica com modelo compacto para ESP32-S3

1. Introdução

A agricultura inteligente, também denominada Smart Farming, combina sensores, conectividade e processamento computacional para acompanhar variáveis do ambiente. Em aplicações de IA embarcada, parte desse processamento pode ser executada diretamente no dispositivo de borda, reduzindo a dependência de serviços externos e permitindo respostas locais.

Neste trabalho foi desenvolvido um modelo de classificação termo-higrométrica utilizando temperatura e umidade do ar como entradas. O fluxo contempla preparação dos dados, comparação de redes neurais compactas, conversão para TensorFlow Lite, quantização INT8 e integração do modelo com um ESP32-S3.

2. Contextualização do problema

Em microcontroladores, a escolha de um modelo não depende apenas da acurácia. Também é necessário observar quantidade de parâmetros, tamanho do arquivo, tipo numérico usado na inferência e facilidade de integração com o firmware. Esses fatores influenciam o uso de memória e a viabilidade da execução na borda.

A aplicação recebe duas grandezas ambientais — temperatura e umidade relativa do ar — e produz uma classe de condição termo-higrométrica. Na etapa embarcada, essas duas variáveis são associadas ao sensor DHT22 simulado no Wokwi, enquanto a inferência é preparada para execução local no ESP32-S3 com TensorFlow Lite Micro.

3. Dataset

Foi utilizado o dataset “Smart Farming Sensor Data for Yield Prediction”, de Atharva Soundankar, disponibilizado no Kaggle. A base possui 500 registros e 22 colunas relacionadas à agricultura inteligente. Para este experimento foram selecionadas apenas as variáveis `temperature_C` e `humidity_%`, que correspondem às entradas usadas pelo modelo.

Como a base já é pequena, os registros foram mantidos sem redução deliberada. Foram verificadas ocorrências de valores ausentes e duplicados, as duas variáveis foram convertidas para formato numérico e registros inválidos foram removidos antes da modelagem.

4. Definição das classes ambientais

Para transformar o problema em uma tarefa de classificação, foi criada a variável `condicao_ambiental`. As classes são didáticas e foram definidas exclusivamente para o experimento da disciplina, não representando diagnóstico ou recomendação agrônômica.

- AMENO: temperatura abaixo de 22 °C;
- QUENTE_SECO: temperatura igual ou superior a 28 °C e umidade abaixo de 65%;
- QUENTE_UMIDO: temperatura igual ou superior a 28 °C e umidade igual ou superior a 65%;
- INTERMEDIARIO: demais combinações de temperatura e umidade.

5. Exploração e preparação dos dados

A exploração dos dados inclui estatísticas descritivas, distribuição das classes, histogramas de temperatura e umidade, gráfico de dispersão temperatura × umidade e estatísticas agrupadas por classe. No notebook atualizado, os gráficos foram padronizados com uma paleta consistente para facilitar a leitura dos comparativos.

As classes foram codificadas com `LabelEncoder`. Os dados foram divididos em treino e teste na proporção de 80% e 20%, com amostragem estratificada. Em seguida, temperatura e umidade foram normalizadas com `StandardScaler`. As médias e os desvios calculados no treino foram preservados para uso na etapa embarcada.

6. Comparação de arquiteturas compactas

Foram avaliadas três redes neurais densas de baixa complexidade, todas com duas entradas e camada de saída `Softmax` para classificação multiclasse.

Arquitetura	Estrutura
A	2 → Dense(4) → saída
B	2 → Dense(8) → saída
C	2 → Dense(8) → Dense(4) → saída

O treinamento utilizou Adam, sparse categorical crossentropy, batch size 16 e `EarlyStopping` monitorando `val_loss`, com restauração dos melhores pesos. As arquiteturas foram comparadas por acurácia, loss, quantidade de parâmetros, número de épocas e tamanho do arquivo `TFLite`.

No teste, a arquitetura A atingiu 86% de acurácia, a arquitetura B atingiu 99% e a arquitetura C atingiu 98%. O aumento de capacidade da arquitetura A para a B trouxe ganho expressivo, enquanto a camada adicional da arquitetura C não melhorou o resultado e aumentou a complexidade.

7. Critério de seleção do modelo

O notebook considera equivalentes os modelos que fiquem até 1 ponto percentual abaixo da melhor acurácia observada. Entre esses candidatos, são priorizados menor número de parâmetros e menor tamanho de arquivo.

A arquitetura B foi selecionada com 60 parâmetros e 99% de acurácia. A arquitetura C ficou dentro da margem de equivalência, porém possui mais parâmetros e não apresentou vantagem prática para a aplicação embarcada.

8. Conversão para TensorFlow Lite e quantização INT8

Após a seleção, o modelo foi convertido para TensorFlow Lite em Float32. Em seguida, foi aplicada quantização inteira INT8 com representative dataset formado por amostras normalizadas do conjunto de treinamento. A conversão força operações, entrada e saída para INT8.

Na inferência quantizada, as entradas são convertidas usando scale e zero point, o modelo é executado e a saída é dequantizada para interpretação. O notebook atualizado também avalia o INT8 por classe com classification report e matriz de confusão, além de comparar diretamente as previsões Float32 e INT8 para medir a concordância entre as duas versões.

9. Resultados do Colab

Os resultados consolidados confirmam que a quantização preservou o desempenho do modelo. A tabela abaixo resume os valores verificados na versão atual do experimento.

Indicador	Float32	INT8
Arquitetura selecionada	B	B
Parâmetros	60	60
Acurácia no teste	99,00%	99,00%
Tamanho do modelo	1,92 KB	2,27 KB
Tipo de entrada/saída	float32	int8

9.1 Avaliação detalhada do INT8

Além da acurácia geral, o notebook gera precision, recall e F1-score por classe e uma matriz de confusão específica para o modelo INT8. Essa verificação ajuda a confirmar que o resultado de 99% não está concentrado em apenas uma classe.

9.2 Concordância Float32 × INT8

Foi acrescentada uma comparação direta entre as previsões das versões Float32 e INT8. A célula calcula a porcentagem de amostras em que as duas versões tomam exatamente a mesma decisão e também informa a variação da acurácia em pontos percentuais. O valor da concordância é produzido automaticamente ao executar o notebook atualizado.

9.3 Interpretação da quantização

Float32 e INT8 atingiram 99% de acurácia, portanto não houve perda de desempenho no teste após a quantização. O arquivo Float32 ficou em aproximadamente 1,92 KB e o INT8 em 2,27 KB. Como a

rede é muito pequena, o overhead do formato TFLite tem peso proporcionalmente maior e, neste caso, o arquivo INT8 não ficou menor. Ainda assim, a versão INT8 é adequada para a etapa embarcada por utilizar operações e tensores inteiros.

10. Implementação embarcada no ESP32-S3 e Wokwi

A integração do modelo no projeto ESP-IDF foi realizada no VS Code. O target foi configurado para esp32s3, o componente esp-tflite-micro foi adicionado ao projeto e os arquivos model_data.cc, model_data.h e model_config.h foram incorporados ao componente principal. O build final foi concluído com sucesso, gerando o firmware para o ESP32-S3.

Para a simulação foi configurado um **ESP32-S3 DevKitC-1** com sensor **DHT22** no Wokwi. As conexões utilizadas são **VCC** → **5V**, **GND** → **GND** e sinal → **GPIO4**. O firmware realiza a leitura das duas entradas, aplica a mesma normalização usada no treinamento, quantiza os valores para INT8 e executa a inferência com TensorFlow Lite Micro.

Até esta atualização, o circuito está configurado e o build está concluído, mas o painel Wokwi Terminal ainda não exibiu a saída serial. Por esse motivo, a evidência final com temperatura, umidade e classe prevista será adicionada assim que a visualização do terminal for normalizada. O relatório não registra uma inferência embarcada que ainda não foi observada no simulador.

11. Discussão

O experimento mostra que uma rede muito pequena pode resolver o problema didático proposto com alto desempenho. A arquitetura B apresentou melhor relação entre resultado e complexidade, enquanto a quantização INT8 manteve a acurácia de 99%. O fato de o arquivo INT8 ter ficado ligeiramente maior que o Float32 reforça a importância de medir os artefatos reais em vez de assumir que a quantização sempre reduzirá o tamanho do arquivo.

Na etapa embarcada, o build bem-sucedido confirma que o modelo INT8 e o TensorFlow Lite Micro foram integrados ao projeto ESP32-S3. A validação completa no Wokwi depende apenas da obtenção da saída serial da simulação para registrar a leitura do DHT22 e a classe calculada pelo modelo.

12. Conclusão

O trabalho completou o pipeline de dados, comparação de arquiteturas, seleção do modelo, conversão para TensorFlow Lite, quantização INT8 e integração do modelo ao projeto ESP32-S3. A arquitetura B, com 60 parâmetros, atingiu 99% de acurácia e manteve esse resultado após a quantização.

A versão Float32 ficou com aproximadamente 1,92 KB e a INT8 com 2,27 KB. Para esta rede muito pequena, a quantização não reduziu o tamanho do arquivo, mas preservou o desempenho e produziu um modelo inteiro adequado ao fluxo de inferência embarcada. O notebook atualizado acrescenta avaliação detalhada do INT8, matriz de confusão e análise de concordância Float32 × INT8. A etapa final de evidência no Wokwi será concluída com a captura da saída serial do ESP32-S3 com o DHT22.